

Python for Web Development

Lesson 9: Deployment to Production

Deployment to Production - Study Notes

Key Concepts

1. **Containerization with Docker** – packaging an application and its dependencies into a portable image.
2. **WSGI server configuration (Gunicorn)** – running a Python web app in production with multiple worker processes.
3. **Platform as a Service deployment (Heroku)** – pushing Docker images or slug builds to a managed environment.
4. **Infrastructure as a Service deployment (AWS)** – using Elastic Container Service (ECS), Elastic Beanstalk, or EC2 to run containers.
5. **Environment specific configuration** – managing secrets, config vars, and scaling settings for production vs. development.

Summary

Deploying a Python web application to production involves turning the codebase into a reproducible artifact (a Docker image), configuring a robust WSGI server such as Gunicorn to handle concurrent requests, and then running that image on a cloud platform that provides scaling, logging, and monitoring.

On **Heroku**, you can either push a Docker image to the Container Registry or let Heroku build a slug from a `Dockerfile` and a `Procfile` that tells the platform how to start Gunicorn. Heroku automatically provisions dynos, handles routing, and provides add ons for databases, logging, etc.

On **AWS**, the same Docker image can be run via **Elastic Beanstalk** (which abstracts EC2/Auto Scaling/ELB) or directly on **ECS/EKS** for more fine grained control over task definitions, service discovery, and IAM roles. In both cases you must set environment variables (secrets, database URLs) and ensure the container exposes the correct port (usually 8000 for Gunicorn).

The production workflow typically looks like:

1. Write a `Dockerfile` that installs dependencies, copies code, and sets the entrypoint to `gunicorn`.
2. Build and test the image locally (`docker build`, `docker run`).
3. Push the image to a registry (Heroku Container Registry, Amazon ECR, or Docker Hub).
4. Deploy the image to the chosen platform, configuring scaling, health checks, and logging.
5. Monitor and iterate – adjust worker count, enable graceful shutdown, and update secrets as needed.

Important Points

- **Dockerfile best practices**
 - Use a slim base image (e.g., `python:3.12-slim`).
 - Install only runtime dependencies; keep build time packages in a separate stage if possible.
 - Set `WORKDIR /app` and copy `requirements.txt` before the rest of the code to leverage Docker layer caching.

- Expose the port Gunicorn will listen on (`EXPOSE 8000`).
- Use `CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:8000", "myapp:app"]` as the final command.
 - **Gunicorn configuration**
- Workers: `workers = 2 \* CPU cores + 1` (adjust based on load testing).
- Worker class: `sync` for typical apps; `gevent` or `eventlet` for async friendly code.
- Timeout: align with your longest request (default 30s).
- Preload application (`--preload`) to reduce memory usage when using many workers.
- Access and error logs: send to stdout/stderr so the platform can capture them.
 - **Heroku specifics**
- `Procfile`: `web: gunicorn -w 4 -b 0.0.0.0:\$PORT myapp:app` (Heroku injects `\$PORT`).
- Add ons: Heroku Postgres, Redis, Papertrail for logs.
- Release phase: `release: python manage.py migrate` (if using Django).
- Scaling: `heroku ps:scale web=2` to run two dynos.
 - **AWS specifics**
- **Elastic Beanstalk**: create a Docker platform application, upload the image or a `Dockerrun.aws.json` file.
- **ECS**: define a task definition with container port 8000, set required IAM role for accessing secrets (Secrets Manager or Parameter Store).
- **Load Balancer**: attach an Application Load Balancer (ALB) to route traffic to the service; configure health check path (e.g., `/health`).
- **Auto Scaling**: set target tracking policy based on CPU utilization or request count per target.
- **Logging**: forward container logs to CloudWatch Logs via the `awslogs` driver.
 - **Security & Configuration**
- Never hard code secrets; use platform provided config vars (Heroku Config, AWS Systems Manager Parameter Store/Secrets Manager).
- Run the container as a non root user (`USER appuser`).
- Keep base images up to date (`docker pull python:3.12-slim` regularly).
- Enable read only filesystem where possible (`docker run --read-only`).

Examples

Example 1: Minimal Dockerfile + Gunicorn Command

```
``dockerfile
```

Use an official slim Python image

```
FROM python:3.12-slim
```

Set working directory

```
WORKDIR /app
```

Install system deps (if needed) and copy requirements

```
RUN apt-get update && apt-get install -y --no-install-recommends gcc && \
  rm -rf /var/lib/apt/lists/*
```

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

Copy application code

COPY . .

Create a non root user

RUN useradd -m appuser

USER appuser

Expose the port Gunicorn will listen on

EXPOSE 8000

Start Gunicorn (4 workers, binding to all interfaces)

CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:8000", "myapp:app"]

...

****Explanation****

- The `slim` image keeps the image size small (~120 /MB).
- System build tools (`gcc`) are installed only to compile any native extensions, then the cache is cleared.
- `requirements.txt` is copied first to maximize Docker layer caching when dependencies haven't changed.
- The app runs as `appuser` to avoid running as root inside the container.
- Gunicorn is launched with 4 workers; the `-b` flag binds to `0.0.0.0:8000` so the platform's router can reach it.

Example 2: Deploying the Same Image to Heroku (Container Stack)

1. ****Create Heroku app****

```
```bash
heroku create my-prod-app
heroku stack:set container # enable Docker container support
```
```

2. ****Log in to Heroku Container Registry****

```
```bash
heroku container:login
```
```

3. ****Build and push****

```
```bash
heroku container:push web --app my-prod-app
heroku container:release web --app my-prod-app
```
```

4. ****Set config vars (e.g., DATABASE_URL)****

```
```bash
heroku config:set DATABASE_URL=postgres://user:pw@host:5432/dbname --app my-prod-app
```
```

5. **Scale dynos**

```
```bash
heroku ps:scale web=2 --app my-prod-app
```
```

Explanation

- Heroku detects the `Dockerfile` and builds the image slug.
- The `web` process type in the `Procfile` (or implicit from `container:push`) tells Heroku how to start the container.
- Config vars are injected as environment variables inside the container, matching the expectations of the app (e.g., reading `os.getenv('DATABASE\_URL')`).
- Scaling to two dynos provides basic redundancy and increased request handling capacity.

Example 3: Deploying to AWS Elastic Beanstalk (Docker Platform)

1. **Create an application**

```
```bash
eb init -p docker my-eb-app --region us-east-1
```
```

2. **Create an environment**

```
```bash
eb create my-eb-env
```
```

